NLP Assignment

Health-Aware / Food Suggestion Agent

-Harmannat Kaur

1. Problem Statement

Giving someone useful food advice is not a task that can be solved with a single prompt. A person with diabetes who has specific ingredients at home and prefers Indian vegetarian cooking needs several things to happen before a useful answer can be produced: their query must be understood and structured, their ingredients must be checked against real nutritional data, unsafe items must be removed, a recipe must be generated from what remains, and that recipe must be evaluated for health suitability before being presented in a friendly way. Each of these is a distinct reasoning step that depends on the output of the previous one. A single prompt asking an LLM to do all of this at once produces vague, hallucinated, and unverifiable answers. The multi-step chain structure forces each step to do one job well, passing structured data forward so the next step can build on it reliably.

2. Chain Design

The agent runs six sequential steps. Each step reads from the shared state dictionary and writes its output back into it. No step can run before the one preceding it without breaking the pipeline.

Step	LLM Call / Tool	Input	Output (fed to next step)
1	LLM — Parse	Raw user text typed by the user	Structured JSON: intent, ingredients, dish_name, dietary_condition, meal_type, diet_type, cuisine
2	Tool — USDA API	Ingredients list + dietary_condition + diet_type from Step 1	Filtered list: safe_ingredients, removed ingredients, real USDA nutrition data per ingredient
3	LLM — Recipe	safe_ingredients OR dish_name + cuisine + diet_type from Step 2	Recipe JSON: recipe_name, ingredients with exact quantities, cooking steps
4	LLM — Nutrition	Recipe (Step 3) + dietary_condition + USDA nutrition data (Step 2)	Nutrition analysis: calories, portion size, safety flag, health note, improvement suggestion
5	LLM — Compile	All outputs from Steps 1, 2, 3, and 4 combined	Single clean final JSON report with all fields merged into one structured object
6	LLM — Respond	Final JSON report from Step 5	Natural language friendly reply displayed to the user

Why each step is separate:

Step 1 exists because the raw user string is unreadable by all downstream steps. Every other step depends on the structured fields this step produces. Step 2 exists because LLMs hallucinate nutritional facts which is the only reliable way to know whether an ingredient is high in sugar or sodium is to ask a real nutritional database. This step acts as a factual gate that the recipe step cannot cross. Step 3 exists because the recipe must be built after filtering.Using the pre-filter ingredient list would defeat the entire health-safety purpose. It also handles three distinct cases: a named dish request, a case where all ingredients were removed, and a case where safe ingredients remain. Step 4 exists because a recipe and a nutritional analysis are different tasks requiring different reasoning. The recipe step creates; the nutrition step evaluates. Merging them would produce a weaker output for both. Step 5 exists because each prior step produces a partial, differently shaped JSON object. This step synthesises all of them into one consistent structure that Step 6 and any downstream system can rely on. Step 6 exists because the final JSON is machine-readable but not appropriate to show a user directly. This step translates structured data into a conversational reply.

3. Tool Integration

The external tool used in Step 2 is the USDA Food Data Central API, available at api.nal.usda.gov. It is a free, publicly available nutritional database maintained by the United States Department of Agriculture. For each ingredient the user provides, the agent sends a GET request with the ingredient name as a search query and receives back nutritional values per 100 grams including calories, sugars, sodium, carbohydrates, and fat. The tool was chosen because LLMs cannot reliably report specific nutritional values. When asked whether a food is safe for a diabetic patient, an LLM will produce a plausible-sounding answer that may be numerically incorrect. The USDA API provides verified figures that the agent uses to make a binary decision: if an ingredient exceeds the defined threshold for the user's condition, it is removed from the safe list. For example, the threshold for sugar in a diabetic diet is set at 5 grams per 100 grams. If the USDA API reports that an ingredient contains 32 grams of sugar per 100 grams, it is flagged and removed before the recipe step receives the ingredient list. The tool output enters the chain in two ways. First, the filtered safe and removed ingredient lists are passed to Step 3, which only generates a recipe using safe ingredients. Second, the raw USDA nutrition data for each ingredient is passed to Step 4, which uses it alongside the recipe to produce a grounded nutritional analysis rather than a hallucinated one. If the API call fails for any ingredient, the agent catches the exception, logs a warning, and treats that ingredient as safe rather than crashing the pipeline. This prevents a network timeout from breaking the entire chain.

4. Limitations

The veg/non-veg filter relies on a hardcoded keyword list, so regional or brand names for meat products will pass through undetected. Cuisine detection covers only Indian or western, meaning requests for Chinese, Mexican, or other cuisines are handled incorrectly. Nutritional thresholds are static approximations and do not account for age, weight, or medical severity. The agent resets state every turn, so follow-up questions about previous recipes will not work.

The Groq free tier causes occasional rate limit errors under rapid usage. Mixed dietary queries are not handled and if a user asks for a vegetarian version of an inherently non-vegetarian dish, the agent attempts it silently rather than flagging the conflict.

5. Reflection

Given more time, the hardcoded veg/non-veg list would be replaced with a food categorisation API, and cuisine support would be expanded beyond two categories. Persistent memory across turns would also be added so the agent can answer follow-up questions correctly. The most useful thing discovered during development was that showing the LLM a complete filled-in JSON example in the Step 1 prompt produced far more reliable structured output than writing the rules in text. The model was returning placeholder text literally until the prompt format was changed.